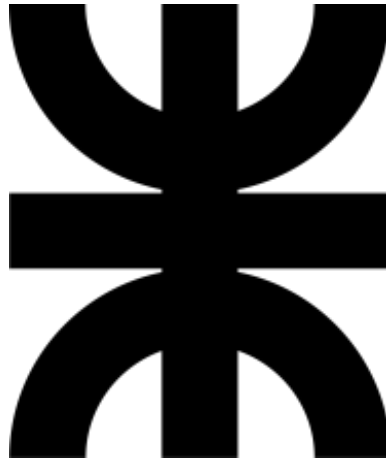


Universidad Tecnológica Nacional

Facultad Regional Córdoba

Ingeniería en Sistemas de Información

Administración de Sistemas de Información



TEMA: TDD

Año: 2026

Docentes

- Salvador Barbera
- Ing. Constanza Garnero

Curso: 4K4

Grupo Nro: 4

Integrantes:

- Alvaretto, Caterina - 95028
- Arrigo, Gaspar - 94135
- Amor, Ignacio - 89507
- Baigorria, Jesica Belen - 401824
- Basly, Agustin - 99238
- Botta Bernaus, Fabrizio - 79553
- Galera, Pedro - 90243
- Nasimbera Falco, Fabrizio Edgar - 96095
- Moyano Serra, Tomas - 96846
- Paná, Matías Aarón - 95165
- Soldera Theaux, Federico - 79554
- Terreno Monla, Florencia Sofia - 90112

Índice

1. Introducción.....	2
2. Decisiones de Diseño.....	3
<i>Elección de Herramientas y Lenguaje.....</i>	<i>3</i>
<i>Resumen del Dominio/Responsabilidades.....</i>	<i>3</i>
<i>Justificación de los Test.....</i>	<i>4</i>
<i>Justificación de Refactorización.....</i>	<i>5</i>
3. Conclusión.....	7

1. Introducción

En este trabajo tenemos como objetivo aplicar los conceptos y principios del Desarrollo Conducido por Pruebas (TDD), el cuál es un enfoque que busca garantizar la calidad del producto desde las primeras etapas del desarrollo, favoreciendo la detección temprana de errores y la construcción de soluciones más robustas, mantenibles y alineadas con los requisitos establecidos.

Para la realización del trabajo utilizamos como base el dominio de negocio “EcoHarmony Park” y para nuestro caso, vamos a desarrollar la historia de usuario “Inscribirme a actividad”, la cual permite a los visitantes registrarse en las distintas actividades ofrecidas por el parque, gestionando la disponibilidad de cupos, verificando los datos de cada visitante y asegurando el cumplimiento de las condiciones particulares de cada actividad.

Utilizamos, por la metodología, el ciclo Red-Green-Refactor. Primero desarrollamos una prueba automatizada que inicialmente falla (Red), luego se implementa el código mínimo necesario para que dicha prueba pase (Green) y finalmente, se realizan mejoras en el diseño y estructura del código sin modificar su comportamiento (Refactor). Este proceso se itera de forma incremental, incorporando un caso de prueba a la vez hasta cubrir la totalidad de los criterios de aceptación definidos en la historia de usuario.

Este enfoque no solo garantiza que cada funcionalidad implementada esté respaldada por pruebas desde su origen, sino que también hay que tener en cuenta que la estructura del código no se planifica por completo de antemano, sino que evoluciona naturalmente a través de los sucesivos ciclos de refactorización, guiada en todo momento por los requisitos expresados en las pruebas.

Como resultado, el trabajo práctico incluye la implementación de la historia de usuario, el conjunto de pruebas automatizadas necesarias para validar tanto los escenarios exitosos como los casos de error, y la justificación de las decisiones de diseño tomadas durante el proceso.

2. Decisiones de Diseño

Elección de Herramientas y Lenguaje

Utilizamos el lenguaje TypeScript junto al framework de testing Jest y el entorno de ejecución Node.js. El justificativo del uso de TypeScript recae en reducir los errores en tiempo de compilación, asegurando que las interfaces de pruebas coincidan con el código de la aplicación. Node.js actúa como entorno de ejecución del servidor y del proceso de pruebas, permitiendo correr código TypeScript fuera del navegador y habilitando el ecosistema de paquetes npm necesario para integrar las herramientas del proyecto.

La elección del framework Jest garantiza que cada archivo de prueba tenga su propio entorno aislado, evitando que compartan estado global, lo que permite ejecutarlos en paralelo de forma fiable. Para agilizar el proceso, Jest ejecuta primero las pruebas que fallaron anteriormente y reorganiza las ejecuciones en función de la duración de los archivos de prueba. Adicionalmente, Jest provee una API de objetos simulados que permite reemplazar dependencias externas, como el servicio de envío de correo electrónico, por implementaciones controladas durante las pruebas, verificando que fueron invocadas con los argumentos correctos sin depender de infraestructura real.

Resumen del Dominio/Responsabilidades

EcoHarmony Park es un bioparque que ofrece a sus visitantes la posibilidad de inscribirse en actividades de una hora de duración. Las actividades disponibles son Tirolesa, Safari, Palestra y Jardinería, cada una con horarios predefinidos y cupos limitados, lo que hace necesaria una gestión controlada del proceso de inscripción.

La responsabilidad central del sistema implementado es permitir que un visitante complete el proceso de inscripción a una actividad de forma íntegra y validada. Para ello, el sistema debe garantizar que la actividad seleccionada tenga cupos disponibles en el horario elegido y que el parque se encuentre operativo en dicho horario. Adicionalmente, debe recopilar los datos personales de cada participante: nombre, DNI y edad de forma obligatoria, y tallas de vestimenta de forma condicional, únicamente cuando la actividad lo requiere, como es el caso de Tirolesa y Palestra.

El proceso de inscripción también exige que el visitante acepte explícitamente los términos y condiciones específicos de la actividad seleccionada, siendo este un requisito sin el cual la inscripción no puede completarse. Una vez que todas las validaciones son satisfechas, el sistema registra la inscripción, descuenta los cupos correspondientes del horario y envía un correo electrónico de confirmación al visitante con los detalles de la inscripción realizada.

El sistema concentra tres grandes responsabilidades: validar las condiciones de disponibilidad y los datos ingresados, registrar la inscripción actualizando el estado del horario, y notificar al visitante mediante el envío del correo de confirmación.

Justificación de los Test

Para validar el correcto funcionamiento de la prueba de usuario, se realizaron 18 pruebas automatizadas siguiendo el enfoque TDD. Estas pruebas fueron diseñadas para cubrir todos los posibles escenarios definidos en los criterios de aceptación y las pruebas de usuario.

Selección de actividad

- Verificación de que el sistema muestra las cuatro actividades disponibles.
- ***Selección de horario***
- Verificación de que solo se muestran horarios con cupos disponibles y con el parque abierto.
- Validación del comportamiento ante la selección de una actividad inexistente.

Inscripciones exitosas

- Inscripción válida en actividades que requieren talle de vestimenta.
- Inscripción válida en actividades que no requieren talle.
- Inscripción válida para múltiples visitantes.

Confirmación de inscripción

- Verificación del envío del correo electrónico de confirmación una vez completada la inscripción.

Validación de cupos y disponibilidad

- Actividad sin cupos disponibles.
- Horario con el parque cerrado.
- Horario inexistente.
- Cantidad de visitantes superior a los cupos disponibles.

Validación de términos y condiciones

- Intento de inscripción sin aceptar los términos y condiciones requeridos.

Validación de talle de vestimenta

- Actividades que requieren talle y no lo reciben como dato de entrada.
- Actividades que no requieren talle y permiten completar la inscripción correctamente.

Validación de datos de visitantes

- Visitante sin nombre.
- Visitante sin DNI.
- Lista de visitantes vacía.

Siguiendo el ciclo Red-Green-Refactor:

La Fase red se puede ver en los primeros commits del proyecto, donde se realiza la creación de pruebas que describen comportamientos esperados del sistema antes de implementar la lógica correspondiente. Por ejemplo tenemos pruebas para los siguientes escenarios:

- debe listar las cuatro actividades disponibles
- debe listar solo los horarios con cupos disponibles y parque abierto
- debe enviar email de confirmación al finalizar la inscripción
- FALLA: actividad sin cupo para el horario seleccionado
- FALLA: no aceptar los términos y condiciones

Cuando estas pruebas fueron creadas inicialmente, todas fallaban porque la funcionalidad aún no había sido desarrollada.

La fase Green se ve después en la implementación de la lógica necesaria para que las pruebas pasaran correctamente. Nuestros test muestran:

- La validación de actividades existentes.
- El control de cupos disponibles.
- La verificación de horarios habilitados.
- El envío del correo de confirmación.
- La validación de términos y condiciones.
- La comprobación de los datos obligatorios de los visitantes.
- La gestión de actividades que requieren o no talle de vestimenta.

La evidencia de esta etapa se encuentra en la ejecución exitosa de los 18 tests automatizados, donde todas las pruebas definidas fueron superadas satisfactoriamente.

Justificación de Refactorización

El ciclo TDD se estructuró en tres etapas iterativas: Red (escritura de una prueba que falla), Green (implementación mínima para que la prueba pase) y Refactor (mejora del diseño sin alterar el comportamiento observable). Las refactorizaciones documentadas a continuación fueron realizadas exclusivamente en la etapa Refactor, verificando en cada caso que la totalidad de las pruebas automatizadas continuará pasando exitosamente.

Eliminar Código Duplicado

En la fase Green, durante la implementación mínima, la lógica de búsqueda de una actividad por nombre y de un horario por identificador se encontraba duplicada en los métodos `inscribir()` y `obtenerHorariosDisponibles()`. Esta duplicación implicaba que los mensajes de error asociados ("Actividad no encontrada" y "Horario no encontrado") podían desincronizarse ante futuras modificaciones. En la refactorización se extrajeron dos métodos privados con responsabilidad única dentro de la clase `ServicioInscripcion`.

La centralización de la lógica de búsqueda y el lanzamiento de errores en un único lugar garantiza consistencia en los mensajes expuestos al sistema y reduce la superficie de cambio ante futuras modificaciones del modelo de datos.

Extracción de Constante

En la fase Green la regla de negocio que determina qué actividades requieren talle de vestimenta podría haberse expresado mediante condicionales explícitos, dispersando el conocimiento de dominio dentro de la lógica de validación. En la fase de refactorización se extrajo una constante nombrada en la capa de entidades. Al centralizar la regla en una constante con nombre semánticamente significativo, se aplica el principio DRY (Don't Repeat Yourself). La incorporación de una nueva actividad con talle requerido implica modificar un único artefacto, sin necesidad de rastrear condicionales distribuidos en el código.

Cada una de las refactorizaciones descritas fue validada ejecutando la suite completa de pruebas automatizadas mediante el comando `npm test`, el cual corre los 29 casos de prueba definidos en `test/inscripcion.test.ts`.

En todos los casos, el resultado fue el esperado: 29 pruebas en estado Green, confirmando que ninguna refactorización alteró el comportamiento observable del sistema, condición necesaria y suficiente para validar la fase Refactor dentro del ciclo TDD.

3. Conclusión

Este trabajo práctico nos dio la oportunidad de aplicar los principios del Desarrollo Conducido por Pruebas (TDD) mediante la implementación de una historia de usuario. A través del ciclo Red-Green-Refactor, el equipo desarrolló una funcionalidad de forma incremental, definiendo primero el comportamiento esperado (pruebas) y luego construyendo el código para llegar a cumplir con ese comportamiento.

La utilización de la metodología TDD nos ayudó a validar continuamente el funcionamiento del sistema, garantizando el correcto desempeño de la funcionalidad respaldada por los tests.

Por otro lado, las etapas de refactorización nos demostraron lo beneficioso de mejorar la estructura interna del código sin modificar comportamiento que ya vimos que funciona, lo cual favorece a la mantenibilidad, reutilización y la claridad al leer el código.

Sin embargo es muy importante ejecutar los test nuevamente después de cada refactorización para comprobar que nuestros cambios no produzcan ningún error en el comportamiento y para brindar confianza en el sistema.

En resumen, este trabajo nos dio un gran conocimiento práctico sobre los beneficios de TDD en el aseguramiento de la calidad, destacando como las pruebas automatizadas pueden guiar la forma en que desarrollamos software, reducir el manifiesto de defectos y facilitar la evolución/mejora del código.